

Contents

1	Introduction	1
2	Related Works	4
3	Preliminaries	5
4	Can CoT improve the Representation Power of RNNs?	7
4.1	CoT Strictly Improves RNNs	7
4.2	CoT Cannot Close the Representation Gap with Transformers	7
4.2.1	Simple Problems on In-Context Retrieval	7
4.2.2	Understanding the Representation Power of RNNs Beyond Simple In-context Retrieval Problems	8
4.2.3	Transformers are Strictly More Expressive Than RNNs	10
5	Enhancing the In-context Retrieval Capability Closes the Representation Gap	10
5.1	Explicit Retrieval Through Regular Expression	11
5.2	Implicit Retrieval by Appending Just One Transformer Layer	12
6	Empirical Validation	13
6.1	Validation on Synthetic Task: IsTree	13
6.2	Validation on Hotpot-QA	13
7	Conclusion and Discussion	14
A	Additional Definitions	21
A.1	Reasoning Tasks on Graphs.	21
A.2	More on Numeric Precisions.	21
A.3	Models	21
A.4	Language Models for Reasoning.	22
B	Omitted Experiment Details	23
C	Omitted Proof	23
C.1	Building Blocks of FFNs Construction	23
C.2	Building Blocks of Transformers Construction	24
C.3	Building Blocks of RNNs Construction	31
C.4	Proof of Theorem 4.6	32
C.5	Proof of Theorem 4.1	33
C.6	Proof of Theorem 4.7	33
C.6.1	Proof of Lemma C.26	34
C.6.2	Proof of Lemma C.27	34
C.7	Proof of Theorem 4.8	36

C.8 Proof of Theorem 5.2	38
C.9 Proof of Theorem 5.3	38
C.10 Proof of Theorem 5.4	40
C.11 Proof of Theorem 5.6	41
C.12 Proof of Theorem 5.7	41
C.13 Proof of Theorem 5.8	41

A Additional Definitions

We will now define some definitions used in the proofs.

A.1 Reasoning Tasks on Graphs.

When reasoning on graphs, without otherwise specified, we will use n as the number of vertices and m as the number of edges. Without loss of generality, we will assume the vertices are labeled by $[n]$.

We will focus on decision problems on graphs, which are defined as follows:

Definition A.1 (Decision Problem on Graphs). A decision problem on graphs is a function $f : \mathcal{G} \rightarrow \{\text{YES}, \text{NO}\}$, where $\mathcal{G}$ is the set of all possible graphs.

We will use the following decision problem as our main example:

Definition A.2 (IsTree). $\text{IsTree}(G) = \text{YES}$ if G is a tree, and $\text{IsTree}(G) = \text{NO}$ otherwise.

One can view IsTree as a minimal example of reasoning tasks. One of the classical solutions to IsTree is running Depth First Search and this algorithm takes $O(n)$ time.

A.2 More on Numeric Precisions.

We will use $\text{ROUND}(x, p)$ to denote the rounding function that rounds x to the nearest number in $\mathbb{R}_p$. We will assume p is an odd number without loss of generality.

$$\mathbb{R}_p = \left\{ (2b_p - 1) \left(\sum_{i=1}^{p-1} b_i 2^{(p-1)/2-i} \right) : \forall i \in [p], b_i \in \{0, 1\} \right\}. \quad (5)$$

For calculation over $\mathbb{R}_p$, we will assume the calculation is exact and the result is rounded to $\mathbb{R}_p$ at the end, that is, for operator $\oplus$, we will have

$$\begin{aligned} & \text{ROUND}(x, p) \oplus_p \text{ROUND}(y, p) \\ &= \text{ROUND}(\text{ROUND}(x, p) \oplus \text{ROUND}(y, p), p). \end{aligned}$$

We will additionally define $\mathbb{Z}_p$ as the set of all integers that can be represented by p -bit floating point numbers. We will define $1/[m]$ as the set of unit fractional $\{\frac{1}{i}\}_{i \in [m]}$. Further, we will define $\text{ROUND}(1/[m], p)$ as the rounding of $1/[m]$ to $\mathbb{R}_p$. We will additionally define for any real number $x \in \{0, 1\}$, $\text{next}(x) = \frac{1}{m+1}$ where $m = \arg \min_{k \in \mathbb{Z}} |x - \frac{1}{k}|$.

A.3 Models

Tokenization. To tokenize a string, we will tokenize all the words separated by the space character into a sequence of tokens. To tokenize a graph G , we will order its edges $E = \{(u_i, v_i) \mid u_i < v_i\}$ randomly and tokenize it into the following string:

$$\text{Tokenize}(G) = \{<s>, u_1, \sim, v_1, \dots, u_m, \sim, v_m\}. \quad (6)$$

We hereby assume there are constant more special tokens that are not the same as any number token, which are listed below:

- $<s>$: the first special token, indicating the start of a sentence.
- $\sim$: the second special token, indicating an edge.
- YES: the third special token, indicating the answer is yes.
- NO: the fourth special token, indicating the answer is no.
- $<\text{StartSearch}>$: the fifth special token, indicating the start of a search query.
- $<\text{EndSearch}>$: the sixth special token, indicating the end of a search query.

We will denote the total number of special tokens as n_S and the total vocabulary size as $|V| = n + n_S$. We will further define the detokenization function Detokenize ,

$$\text{Detokenize}(\mathcal{S}) = \mathcal{S}_1 \mathcal{S}_2 \dots \mathcal{S}_l.$$

Here each $\mathcal{S}_i$ is either a number or a special token, which we will treat as a word.